APP.PY LINE-BY-LINE EXPLANATION

app.py is the main Flask application. It configures the web app, handles login/signup, phone OTP verification, optional Google OAuth, protected pages, JSON APIs, inquiry submission, and the 404 page.

IMPORTS

Lines 1-5: import standard helpers for environment variables, secure random OTPs, time checks, decorators, and URL parsing/building.

Lines 7-16: import Flask objects used for routes, templates, sessions, redirects, API JSON responses, and flashed messages.

Line 17: imports password-hash helpers so OTP values can be stored and checked as hashes instead of plain text.

Lines 19-26: import local auth helpers from auth_store.py for finding, normalizing, exposing, and creating users.

Line 27: imports database helpers from db.py for environment loading, listings, inquiries, and the storage label.

Lines 29-32: tries to import Authlib OAuth. If unavailable, Google sign-in is disabled without breaking the app.

APP SETUP

Line 35: loads .env values into the process before app config is read.

Line 37: creates the Flask application object.

Line 38: sets SECRET_KEY from the environment, with a development fallback.

Lines 39-41: configure session cookie behavior and default URL scheme.

Lines 44-48: _env_int safely reads an integer environment value and falls back if it is missing or invalid.

Lines 51-52: configure OTP lifetime and whether local demo OTP mode is enabled.

GOOGLE OAUTH SETUP

Lines 54-55: create the OAuth registry if Authlib is installed and initialize the Google client variable.

Lines 57-70: register the Google OAuth client only when GOOGLE_CLIENT_ID and

GOOGLE_CLIENT_SECRET are present.

Lines 62-69: define Google's authorize/token/userinfo endpoints, issuer, signing algorithm, and requested scopes.

REQUEST HELPERS

Lines 73-80: PUBLIC_ENDPOINTS lists routes that do not require an existing login.

Lines 83-91: _filters_from_request collects search filters from query-string parameters and strips whitespace.

Lines 94-95: _current_user loads the session user id and converts the full user record into public-safe data.

Lines 98-101: _normalize_next_url accepts only same-site relative next URLs and otherwise falls back to a named endpoint.

Lines 104-108: _safe_next resolves the next destination from form/query values or a saved OAuth next URL.

Lines 111-113: _login_user clears the session and stores the authenticated user's id.

Lines 116-117: _google_enabled returns whether Google OAuth is configured.

GOOGLE URL HELPERS

Lines 120-123: _google_redirect_uri returns GOOGLE_REDIRECT_URI or builds the callback URL from Flask routing.

Lines 126-128: _google_redirect_host extracts the hostname and port from the redirect URI.

Lines 131-148: _canonical_local_url redirects 127.0.0.1:5000 to localhost:5000 when the Google redirect URI expects localhost.

PHONE OTP FLOW

Lines 151-152: _generate_otp creates a zero-padded six-digit OTP.

Lines 155-159: _start_phone_otp normalizes and validates the submitted phone number.

Lines 161-167: enforce login/signup rules so login requires an account and signup rejects an existing phone number.

Lines 169-174: stop phone OTP when demo mode is disabled and no SMS provider is configured.

Lines 176-184: generate the OTP and store a hashed version, phone, name, mode, expiry, and

next URL in the session.

Lines 186-189: store and flash the demo OTP, then redirect to the verification page.

AUTHENTICATION GUARDS

Lines 192-203: login_required wraps protected views; API requests get 401 JSON while browser pages redirect to login.

Lines 206-218: require_login runs before each request, handles localhost canonicalization, allows public/API storage endpoints, and blocks unauthenticated access.

Lines 221-226: inject_auth_context makes current_user and google_oauth_enabled available to every template.

LOGIN AND SIGNUP ROUTES

Lines 229-241: /login shows the login form and starts the phone OTP login flow on POST.

Lines 244-260: /signup shows the signup form, requires a name, and starts the phone OTP signup flow on POST.

Lines 263-269: /verify-otp redirects users back to login if no OTP flow is pending.

Lines 270-285: on OTP POST, reject expired codes and re-render the form when the code hash does not match.

Lines 286-300: create/update the phone user, clear OTP session data, log in the user, flash success, and redirect.

Lines 302-306: on GET, render the OTP form with pending data and demo OTP if available.

GOOGLE ROUTES

Lines 309-316: /auth/google checks configuration, stores the safe next URL, and sends the user to Google’s OAuth screen.

Lines 319-337: /auth/google/callback validates configuration, exchanges the OAuth code for a token, loads userinfo, and reports failures.

Lines 339-354: require a usable Google subject/email, upsert the local Google user, log in, and redirect.

Lines 357-361: /logout clears the session, flashes a sign-out message, and returns to login.

PAGES AND APIS

Lines 364-368: / is the protected home page and passes current filters to index.html.

Lines 371-374: /property/<listing_id> renders the protected property detail shell.

Lines 377-379: /api/storage returns whether data is coming from demo data or MySQL.

Lines 382-387: /api/listings applies filters and returns matching listings as JSON.

Lines 390-396: /api/listings/<listing_id> returns one listing or a 404 JSON error.

Lines 399-427: /inquiry validates the form, saves or stores the inquiry, flashes the result, and redirects back to the property.

Lines 430-432: render 404.html for missing pages.

Lines 435-436: run the Flask dev server when app.py is executed directly.